

LAB RESOURCE UTILIZATION PLATFORM

Weeks5& 6

Team Name	Four
Team Members	Vinay Sahu, Aman Tiwari, Chinta Dharani
Milestone	Milestone 3 (Week 5 & 6)
Document Type	Technical & Functional Documentation

Table of Contents

1. Project Overview
2. Introduction
3. Milestone 3 Objectives
4. Technologies Used
5. System Architecture
6. Features Implemented
7. Backend Implementation
8. Frontend Implementation
9. Workflow — How the Milestone 3 Modules Connect
10. Challenges Faced
11. Conclusion

1. Project Overview

The Lab Resource Utilization Platform is a web-based system built to help academic institutions manage, monitor, and optimize the usage of shared laboratory equipment. It brings equipment booking, maintenance, calibration, certification tracking, cost accounting, and analytics into a single role-aware application, replacing manual spreadsheets and disconnected processes with a live, auditable system.

The platform is built as a two-tier web application: a Spring Boot REST API backend backed by a PostgreSQL database, and a React single-page application frontend. Access is secured with JWT-based authentication and enforced per endpoint through Spring Security role checks, so every screen and every API call respects the signed-in user's role.

Milestone Focus

Milestone 3 (Weeks 5 & 6) extends the platform delivered in earlier milestones with four connected capability areas: equipment Maintenance & Work Orders, Calibration & Certification tracking, Cost Management & inter-institution billing, and Role-Based Analytics — tied together by a scheduled Notification & Alert system.

2. Introduction

This document describes the design and implementation of Milestone 3 of the Lab Resource Utilization Platform, developed as part of an internship project. It is intended to give a mentor or reviewer a clear, code-grounded picture of what was built, how the pieces fit together, and how each module can be verified against the source code.

2.1 Team Information

Field	Details
Team Name	Four
Team Members	Vinay Sahu, Aman, Dharni

3. Milestone 3 Objectives

Based on the implemented source code, Milestone 3 targeted the following objectives:

- Introduce structured equipment maintenance tracking with technician assignment and status progression (Scheduled → Active → Completed).
- Add calibration and certification tracking per piece of equipment, including due-soon and overdue/expired detection.
- Automate usage-based cost calculation for equipment bookings and allocate those costs to departments, forming the basis of inter-institution billing.

- Provide role-based analytics dashboards that combine utilization, cost, and booking data into a single aggregated view for managers and administrators.
- Generate utilization and cost reports that summarize platform activity for oversight roles.
- Deliver a scheduled notification and alert system that proactively informs technicians, managers, and students about calibration/certification deadlines, maintenance due dates, idle equipment, and upcoming bookings.
- Connect all of the above into one continuous operational workflow, from equipment upkeep through to cost visibility and alerting.

4. Technologies Used

The technology choices below are taken directly from the project's build configuration and package manifests.

4.1 Backend

Category	Technology
Language / Runtime	Java 17
Framework	Spring Boot 4.1.0
Web Layer	Spring Web MVC (spring-boot-starter-webmvc) Spring
Security	Security + JWT (io.jsonwebtoken / jjwt 0.12.6) Spring
Data Access	Data JPA / Hibernate
Database	PostgreSQL
Real-Time	Spring WebSocket (STOMP) spring-boot-starter-
Validation	validation
Build Tool	Gradle (Groovy DSL)
Testing	JUnit 5 (spring-boot-starter-*test modules)

4.2 Frontend

Category	Technology
Library	React 19.2
Build Tool	Vite 8
Routing	react-router-dom 7
HTTP Client	Axios / Fetch API

Category Charts	Technology
Real-Time Client	Recharts 3
Styling	@stomp/stompjs + sockjs-client
Linting	Component-scoped CSS files
	ESLint 10

5. System Architecture

The platform follows a layered client-server architecture. The React frontend communicates with the Spring Boot backend exclusively over a REST/JSON API secured with JWT bearer tokens, plus a WebSocket (STOMP over SockJS) channel used for real-time notification delivery. The backend itself follows the standard Controller → Service → Repository layering on top of a PostgreSQL database.

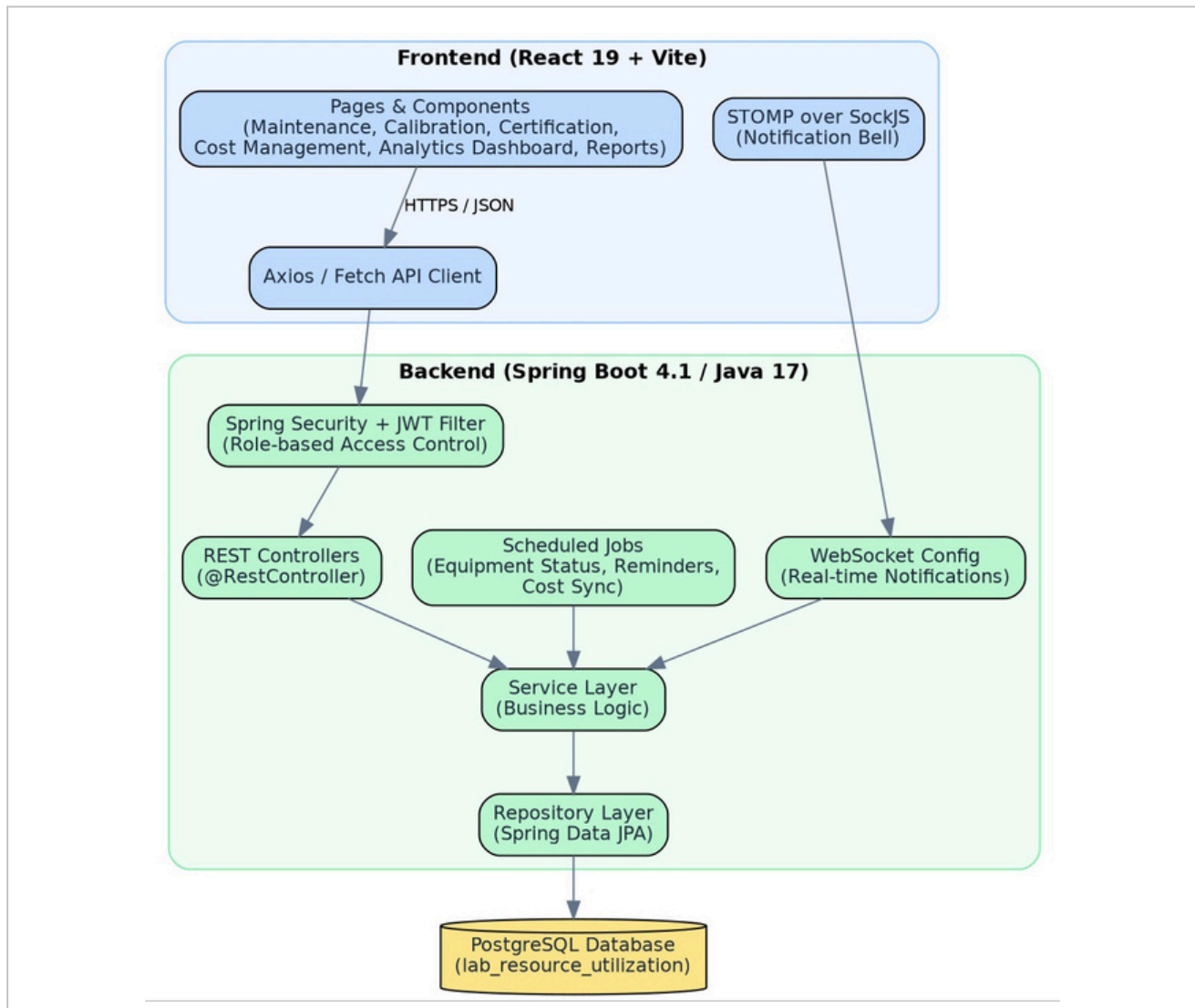


Figure 1: High-level system architecture of the Lab Resource Utilization Platform

5.1 Backend Layering

- **Controllers** — expose REST endpoints under `/api/**`, apply role checks via `@PreAuthorize`.
- **Services** — contain business logic (e.g., cost calculation, status transitions, notification de-duplication).
- **Repositories** — Spring Data JPA interfaces for database access.

- Entities — JPA-mapped domain objects (Equipment, Maintenance, EquipmentCalibration, EquipmentCertification, EquipmentUsageCost, DepartmentCostAllocation, Notification, etc.).
- Schedulers — background jobs that keep equipment status, costs, and notifications in sync without manual intervention.

5.2 Security Model

Authentication is stateless and JWT-based: a `JwtAuthenticationFilter` validates the bearer token on every request and populates the Spring Security context, while `SecurityConfig` enables method-level security (`@EnableMethodSecurity`) so each controller method independently declares which roles may call it (e.g., `hasAnyRole('LAB_MANAGER','DEPARTMENT_HEAD','INSTITUTION_ADMIN','SYSTEM_ADMIN')`).

6. Features Implemented

The features below are described strictly from what exists in the uploaded backend and frontend source code — corresponding controller endpoints, service logic, and frontend pages are cited for each.

6.1 Maintenance & Work Order Management

Maintenance records are tracked per equipment item with a status lifecycle, a scheduled/next maintenance date, and an assigned technician.

- Backend: MaintenanceController exposes list, get-by-id, create, update, and a “My Tasks” endpoint (/api/maintenance/my-tasks) that returns work orders assigned to the logged-in technician.
- A separate WorkOrder entity (linked to a MaintenanceRequest, Equipment, and assigned technician) models formal work orders with priority, scheduled date, and completion date.
- EquipmentStatusScheduler automatically activates “Scheduled” maintenance once its date arrives, and puts equipment into an “Under Maintenance” status while active maintenance exists for it.
- Frontend: the Maintenance page (Maintenance.jsx) provides technicians and managers a management console for these records, restricted to LAB_TECHNICIAN, LAB_MANAGER, DEPARTMENT_HEAD, INSTITUTION_ADMIN, and SYSTEM_ADMIN roles.

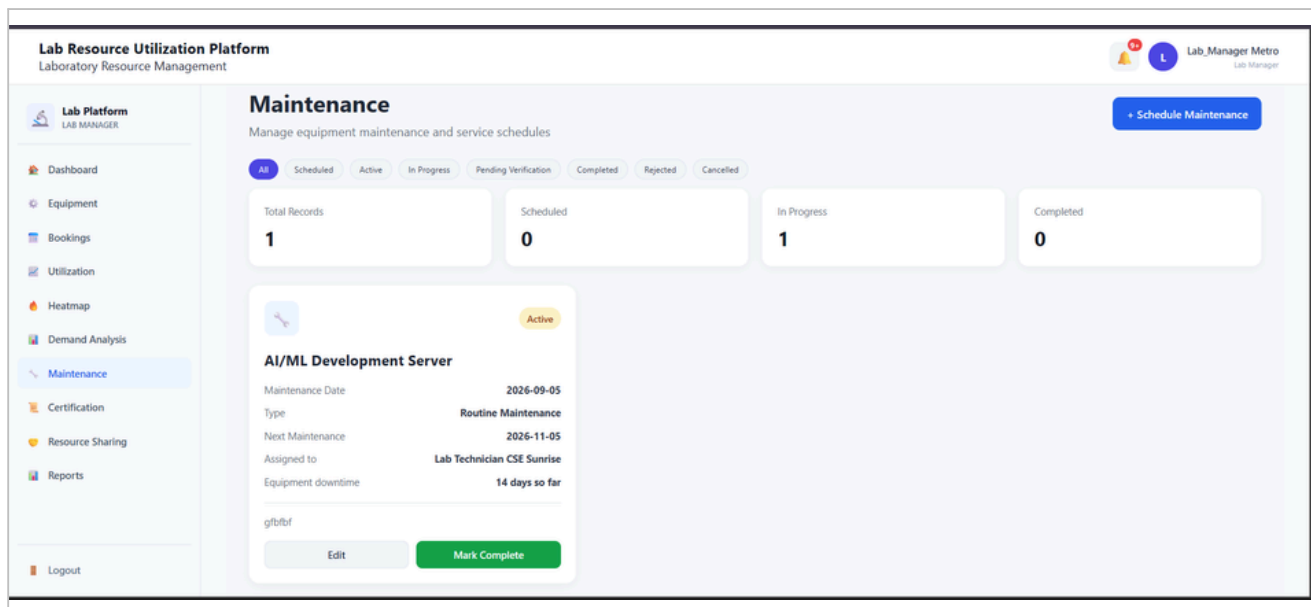


Figure: Maintenance module — Lab Manager view

6.2 Calibration & Certification

Two related but distinct modules track equipment fitness for use.

- Calibration: EquipmentCalibration records calibration date, next-due date, calibrated-by, status, and certificate number. CalibrationController exposes /due-soon and /overdue endpoints in addition to standard CRUD.

- Certification: EquipmentCertification records certification name, issuing authority, issue/expiry dates, and status, with /expiring-soon and /expired endpoints on CertificationController.
- Both modules expose a read-only per-equipment history endpoint that is also visible to STUDENT accounts, so learners can check whether a machine is currently trustworthy before booking it.
- Frontend: dedicated Calibration.jsx and Certification.jsx pages, each reachable via the sidebar for technical/managerial roles.

Lab Resource Utilization Platform
Laboratory Resource Management

Calibration & Certification
Track calibration cycles and certification renewals

Log New Calibration

Equipment: Drilling Machine

Calibration Date: 19-09-2026

Next Calibration Date: 19-10-2026

Certificate Number: CERT-005

Status: **COMPLETED** SCHEDULED IN PROGRESS CANCELLED

Remarks: Drill bit was changed

Buttons: Cancel Log Calibration

Status	Calibrated By	Actions
COMPLETED	shubham	Edit
COMPLETED	Lab_Technician metro	Edit
COMPLETED	Lab Technician CSE Sunrise	Edit
COMPLETED	Lab Technician CSE Sunrise	Edit

Figure: Calibration module — Log New Calibration (Lab Technician)

Lab Resource Utilization Platform
Laboratory Resource Management

Certification Tracking
Track certificate validity and renewals

Add Certification

Equipment: Function Generator

Certification Name: Fitness Certificate

Certificate Number: CERT-005

Issued Date: 19-09-2026

Expiry Date: 19-10-2026

Issuing Authority: Lab Manager

Remarks: Active Now

Buttons: Cancel Add Certification

Expiry	Authority	Status	Actions
2026-10-04	Lab Manager	ACTIVE	Edit
2026-10-04	Lab Manager	ACTIVE	Edit
2026-10-05	Lab Manager	ACTIVE	Edit
2026-09-06	Laboratory administration	ACTIVE	Edit

Figure: Certification module — Add Certification (Lab Manager)

6.3 Cost Management & Inter-Institution Billing

Usage-based costing turns completed bookings into billable records automatically.

- EquipmentUsageCost captures usage start/end, usage hours, the equipment's hourly rate, and the resulting total cost per booking, with a PENDING / PAID / WAIVED cost_status.
- DepartmentCostAllocation links each usage cost to the requesting Department, forming the basis for allocating shared-equipment costs across departments and institutions. CostManagementController exposes a cost list, a /summary dashboard (totals, cost by equipment/department, monthly trend), a manual /generate re-sync, a cost-status update endpoint, and an equipment hourly-rate update endpoint — all restricted to manager/admin roles. EquipmentStatusScheduler calls generateMissingCostRecords() on every scheduled run, so as soon as a booking auto-completes it is turned into a usage-cost record without manual action. Resource sharing between institutions (ResourceSharingRequest, with sender/receiver Institution references) provides the cross-institution booking pathway whose resulting usage is then costed and allocated the same way.
- Frontend: CostManagement.jsx lets eligible roles view records, trigger the manual generate action, update cost status, and edit equipment billing rates.

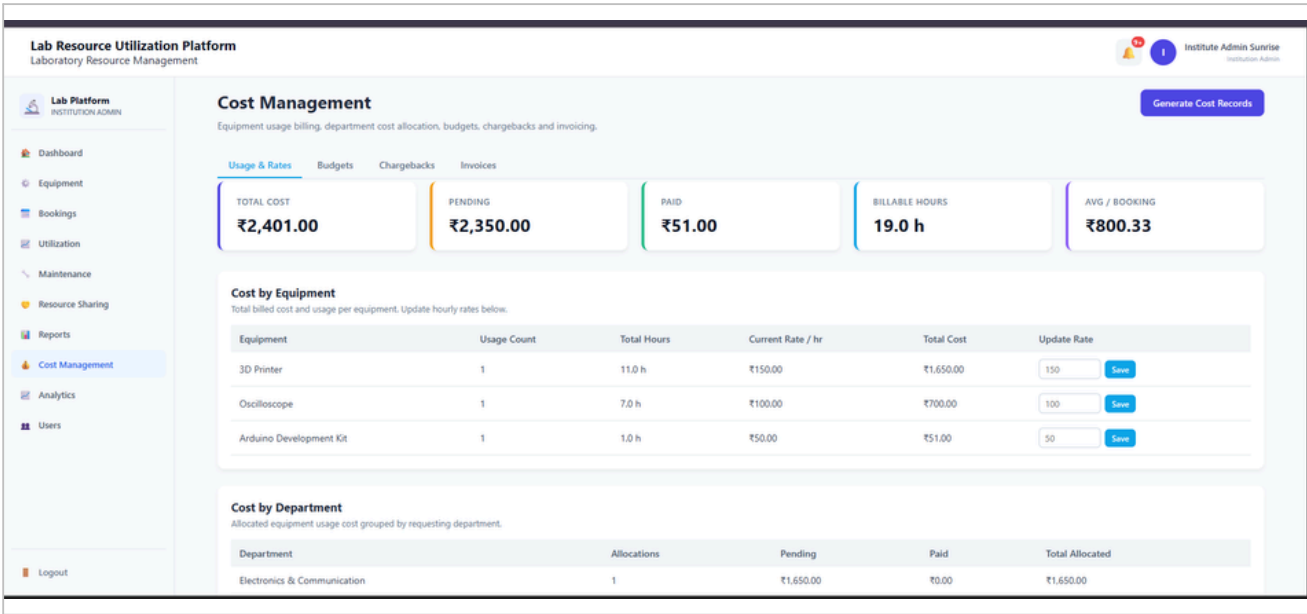


Figure: Cost Management module — Usage & Rates (Institution Admin)

6.4 Role-Based Analytics Dashboards

A single aggregated analytics endpoint feeds a dashboard built for managers and above.

- ~~LAB MANAGER~~ **AnalyticsDashboardDTO** combines utilization figures, cost figures, and booking trends from **DEPARTMENT HEAD**. **INSTITUTION_ADMIN**, **SYSTEM_ADMIN**) returns **AnalyticsServiceImpl**
- and **CostManagementService** and additionally computes booking status **UtilizationService** and monthly booking trends from **BookingRepository**.

- Institution-scoped visibility: non-SYSTEM_ADMIN users only see analytics for their own institution, enforced in the service layer via the authenticated user's institution ID.
- Frontend: AnalyticsDashboard.jsx renders this payload with Recharts (bar, line, and pie charts) for utilization, cost, and booking-status views.

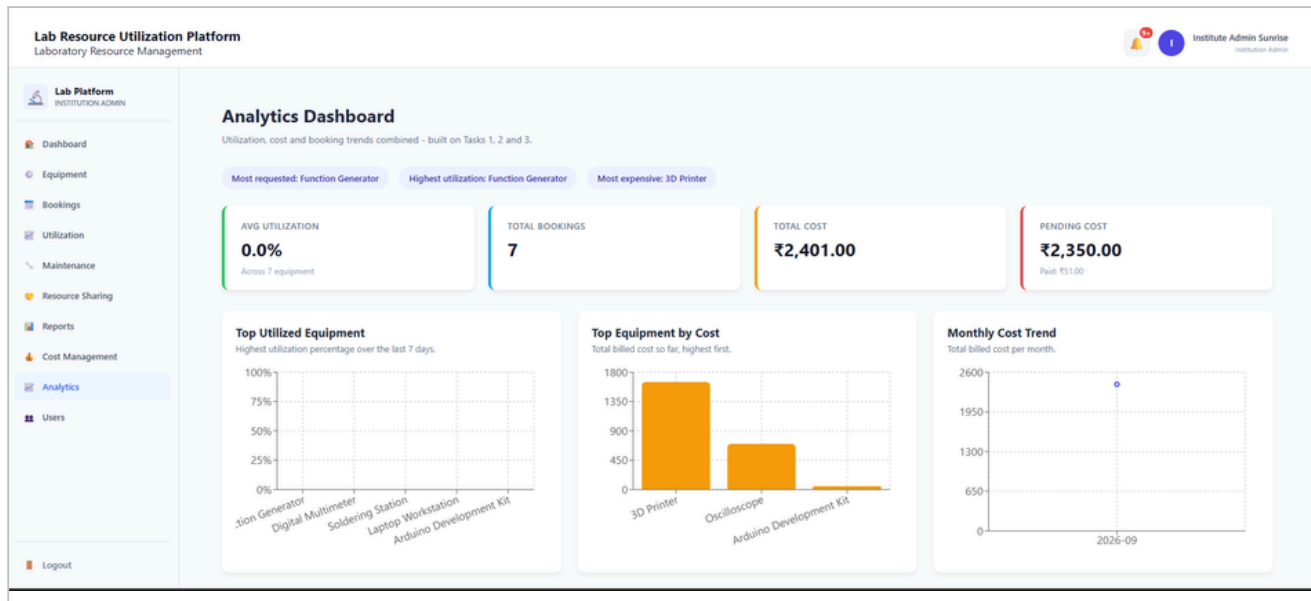


Figure: Analytics Dashboard — Institution Admin view

6.5 Utilization & Cost Reports

Beyond the analytics dashboard, the platform provides a Reports view aimed at oversight roles.

- `utilizationController` exposes `/api/utilization` (raw per-equipment utilization data) `/api/utilization/summary` (aggregated totals, most-requested/highest/lowest-utilization equipment) to LAB_MANAGER and above.
- `CostManagementController`'s `/api/costs/summary` supplies the cost-reporting figures (totals, pending vs. paid, cost by equipment/department, monthly trend).
- For INSTITUTION_ADMIN/SYSTEM_ADMIN user equipment summary reports, scoped to the viewer's own institution unless they are a SYSTEM_ADMIN.

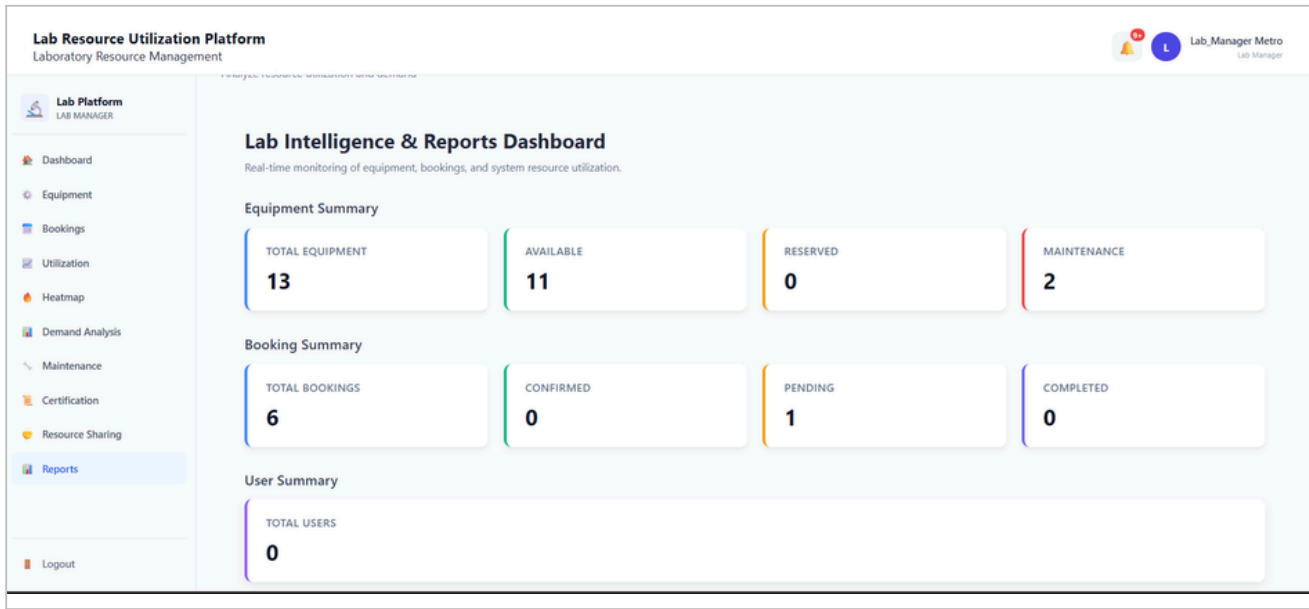


Figure: Lab Intelligence & Reports Dashboard — Lab Manager view

6.6 Notification & Alert System

A scheduled, de-duplicated notification pipeline keeps users informed without manual follow-up.

- NotificationController exposes endpoints to list a user's notifications, list only unread notifications, and mark a notification as read — available to every authenticated role.
- EquipmentStatusScheduler runs a daily reminder sweep (cron 8:00 AM) that checks calibration due-soon/overdue, certification expiring/expired, maintenance due/overdue, and idle-equipment (14+ idle days, manager-only) conditions, and creates notifications via createIfNotAlreadyNotifiedToday to avoid duplicate alerts on the same day.
- A separate 15-minute job (sendBookingReminders) notifies students of bookings starting within the next hour.
- Frontend: NotificationBell.jsx (with the useNotifications hook) renders unread counts and per-type action links (e.g., “Go to Calibration”, “View Booking”), and WebSocketConfig on the backend supports real-time push over STOMP/SockJS.

7. Backend Implementation

The backend follows a conventional Spring Boot

classes implemented for the Milestone 3 modules layered structure. The tables below list the concrete

7.1 APIs / Controllers

Controller	Base Path	Purpose
MaintenanceController	/api/maintenance	Work order CRUD, technician “My Tasks” view
CalibrationController	/api/calibrations	Calibration CRUD, due-soon / overdue queries
CertificationController	/api/certifications	Certification CRUD, expiring-soon / expired queries
CostManagementController	/api/costs	Usage-cost listing, summary, status & rate updates
AnalyticsController	/api/analytics	Combined analytics dashboard payload
UtilizationController	/api/utilization	Per-equipment utilization data & summary
NotificationController	/api/notifications	List / mark-as-read notifications
ResourceSharingController	/api/resource-sharing	Cross-institution equipment sharing requests
WaitlistController	/api/waitlist	Equipment waitlist join / view / cancel

7.2 Services

Business logic sits in a Service interface + *ServiceImpl implementation pair for each domain area, keeping controllers thin. Key services relevant to Milestone 3:

- MaintenanceService / MaintenanceServiceImpl — work order lifecycle and “my tasks” scoping.
- CalibrationService, CertificationService — due/expiry calculations.
- CostManagementService / CostManagementServiceImpl — usage-cost generation, summary aggregation, status & rate updates.
- AnalyticsService / AnalyticsServiceImpl — composes utilization + cost + booking data into one dashboard DTO, with institution-scoped visibility.
- NotificationService / NotificationServiceImpl — notification creation with same-day de-duplication (createIfNotAlreadyNotifiedToday).
- UtilizationService / UtilizationServiceImpl — raw and summarized utilization metrics.

7.3 Repositories

Data access uses Spring Data JPA repositories, one per entity, including `MaintenanceRepository`, `CalibrationRepository`, `CertificationRepository`, `EquipmentUsageCostRepository`, `DepartmentCostAllocationRepository`, `NotificationRepository`, `BookingRepository`, `EquipmentRepository`, and `UserRepository`, among others. These provide derived-query methods used by the scheduler, e.g. `findByNextCalibrationDateBetween`, `findByExpiryDateLessThanEqual`, and `findByMaintenanceStatus`.

7.4 Database Integration

The application connects to a PostgreSQL database (`lab_resource_utilization`) via Spring Data JPA / Hibernate, with `spring.jpa.hibernate.ddl-auto=update` so entity changes are reflected in the schema automatically during development.

Core Milestone 3 tables (mapped from JPA entities) include: `maintenance`, `work_orders`, `equipment_calibrations`, `equipment_certifications`, `equipment_usage_costs`, `department_cost_allocations`, and `notifications` — each linked back to `equipment`, `users`, `departments`, and `institutions` through foreign-key relationships (`@ManyToOne`).

8. Frontend Implementation

8.1 Pages

Page	Route	Milestone 3 Relevance
Maintenance.jsx	/maintenance	Work order tracking & technician tasks
Calibration.jsx	/calibration	Calibration logging & due/overdue views
Certification.jsx	/certification	Certification logging & expiry views
CostManagement.jsx	/cost-management	Usage-cost records, summary, rate & status editing
AnalyticsDashboard.jsx	/analytics	Combined utilization/cost/booking charts
Reports.jsx	/reports	Institution-scoped summary reporting
Utilization.jsx / Heatmap.jsx / DemandAnalysis.jsx	/utilization, /heatmap, /demand-analysis	Supporting utilization views feeding analytics
ResourceSharing.jsx	/resource-sharing	Cross-institution equipment sharing requests

8.2 Components

- Sidebar.jsx / Navbar.jsx / MainLayout.jsx — role-aware navigation shell that shows or hides each module link based on the signed-in user's role.
- NotificationBell.jsx + useNotifications hook — polls/receives notifications, shows unread counts, and routes the user to the relevant module (Calibration, Waitlist, Reservations, etc.) per notification type.
- BookingForm.jsx, BookingList.jsx, BookingApproval.jsx, BookingStatus.jsx, BookingAvailability.jsx — booking components whose completed bookings feed the Cost Management module.
- ShareableEquipment.jsx, IncomingShareRequests.jsx, OutgoingShareRequests.jsx, ResourceShareRequestForm.jsx — cross-institution resource sharing UI.

8.3 User Roles

Six roles are enforced consistently across both frontend route guards (ProtectedRoute + Sidebar visibility checks) and backend @PreAuthorize annotations:

Role	Milestone 3 Access
STUDENT	Read-only calibration/certification history on equipment; notifications; waitlist; bookings.
LAB_TECHNICIAN	Maintenance, Calibration, Certification — full logging and “My Tasks” view.
LAB_MANAGER	All technician access plus Cost Management, Analytics, Reports, Resource Sharing.
DEPARTMENT_HEAD	Same access tier as Lab Manager for Milestone 3 modules.
INSTITUTION_ADMIN	Full institution-scoped access, including User management.
SYSTEM_ADMIN	Full cross-institution (platform-wide) access to every module.

8.4 Navigation Flow

After JWT-based login, ProtectedRoute checks the user's role (stored in sessionStorage) against each route's allowedRoles list before rendering MainLayout, which wraps every page with the Sidebar and Navbar. The Sidebar independently re-checks role permissions to decide which links to display, so navigation options always match what the backend will actually authorize.

9. Workflow — How the Milestone 3 Modules Connect

The six Milestone 3 modules are not independent features; they form one continuous operational loop, driven largely by the scheduled jobs in EquipmentStatusScheduler:

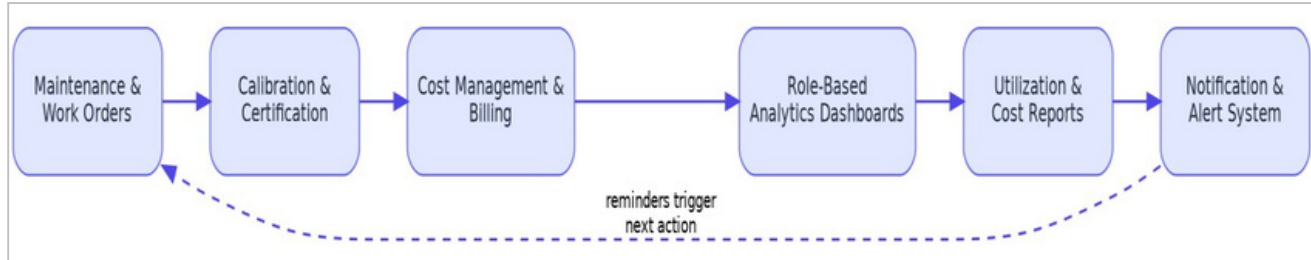


Figure 2: Milestone 3 module workflow

- **Maintenance** — a maintenance record becomes due; the scheduler marks it Active and the equipment status flips to “Under Maintenance”, blocking new bookings.
- **Calibration & Certification** — technicians log calibration/certification results for the serviced equipment; the daily sweep watches next_calibration_date and expiry_date to flag upcoming or missed deadlines.
- **Cost Management** — as bookings for the (now available) equipment complete, the scheduler's generateMissingCostRecords() step turns them into EquipmentUsageCost records and allocates the cost to the requesting department (DepartmentCostAllocation), including cross-institution usage originating from Resource Sharing.
- **Analytics** — AnalyticsServiceImpl pulls the resulting utilization and cost figures together into one dashboard payload for managers/admins.
- **Reports** — the same underlying utilization and cost summaries are surfaced as institution-scoped reports for oversight roles.
- **Notifications** — throughout the loop, the daily and 15-minute scheduled jobs generate de-duplicated alerts (calibration due/overdue, certification expiring/expired, maintenance due/overdue, idle equipment, booking reminders) that route users back to the module that needs attention — closing the loop back to Maintenance.

10. Challenges Faced

The following challenges are evident from comments and fixes left in the source code itself:

- Institution-scoped data leakage: several pages (Reports.jsx, AnalyticsServiceImpl) needed explicit filtering because certain endpoints (e.g., /api/equipment) intentionally return cross-institution data for booking purposes, but reports/analytics needed to show only the viewer's own institution unless they are a SYSTEM_ADMIN.
- Role-permission mismatches: Reports.jsx originally called the admin-only /api/users endpoint ~~for~~ conditionally, causing 403 errors and a stuck “0” user count LAB_MANAGER/DEPARTMENT_HEAD users — resolved by gating that call behind a canViewUsers role check.
- Avoiding duplicate/noisy alerts: the notification scheduler needed a same-day de-duplication guard (createIfNotAlreadyNotifiedToday) so daily and 15-minute jobs don't spam users with repeat reminders.
- Correcting a data-source bug: certification expiry reminders were at one point read from the calibration table; the scheduler was corrected to source them from the real EquipmentCertification module instead (documented in-code as a “Task 2 audit fix”).
- Keeping cost records in sync automatically: rather than requiring a manual step, usage-cost generation had to be wired into the periodic scheduler (generateMissingCostRecords) so newly completed bookings are always billed without staff intervention, while still exposing a manual /generate endpoint as a safety net.

11. Conclusion

Milestone 3 successfully extends the Lab Resource Utilization Platform with a connected set of operational modules: equipment maintenance and work orders, calibration and certification tracking, automated usage-based cost management with department-level allocation, role-based analytics, utilization/cost reporting, and a scheduled notification system. Each module is backed by dedicated JPA entities, Spring Data repositories, service-layer business logic, and role-restricted REST endpoints, with a matching React page and role-aware navigation on the frontend.

Most importantly, the modules are not siloed: the EquipmentStatusScheduler ties maintenance, calibration/certification, cost generation, and notifications into a single recurring operational loop, giving the platform the kind of automated day-to-day upkeep a real multi-institution lab-sharing system needs. This provides a solid foundation for the analytics, reporting, and billing features that later milestones can build further upon.

Prepared by Team Four (Vinay Sahu, Aman, Dharni) for internship mentor review.